

Towards a Constraint-Based Approach for Quantum Circuit Modeling

Daniel San Martín
Escuela de Ingeniería
Universidad Católica del Norte
daniel.sanmartin@ucn.cl

Valter Vieira de Camargo
Departamento de Computação
Universidade Federal de São Carlos
valtervcamargo@ufscar.br

Paul Leger
Escuela de Ingeniería
Universidad Católica del Norte
pleger@ucn.cl

Abstract—Quantum software development remains challenging due to the low-level abstractions typically used to construct quantum circuits, where qubits, gates, and measurements are directly manipulated through programming APIs. As circuit complexity increases, reasoning about structural correctness and detecting modeling errors becomes increasingly difficult. To address this issue, this paper proposes a constraint-based modeling approach for quantum circuits grounded in model-driven engineering principles. A domain-specific metamodel is introduced to represent the essential components of quantum circuits, including qubits, quantum gates, measurements, classical storage, and execution flows. On top of this structural representation, a validation layer based on the KCL (Kusion Configuration Language) constraint language is defined to enforce well-formedness rules governing both structural and behavioral properties of circuit models. Eight constraints are specified to regulate aspects such as valid flow connections, correct qubit references, measurement consistency, and the interaction between quantum operations and classical control information. The approach is evaluated through a feasibility study based on the quantum teleportation protocol, modeled using the proposed metamodel. Validation experiments include both a correct circuit model and faulty variants containing injected modeling errors. The results show that the constraint-based validation mechanism successfully detects inconsistencies in circuit structure and classical–quantum dependencies before execution. These findings suggest that integrating constraint-based validation into quantum circuit modeling can improve early fault detection and contribute to more reliable quantum software development.

Index Terms—Quantum circuit modeling, metamodeling, constraints, validation

I. INTRODUCTION

Despite rapid progress in quantum computing, quantum software development remains difficult and error-prone [1], [2], [3]. In current practice, quantum programs are typically constructed through low-level APIs in which qubits, gates, measurements, and classical registers are manipulated directly in code. Although such frameworks support execution, they offer limited support for high-level reasoning about circuit structure and correctness as complexity increases [4], [5]. This limitation is especially relevant in the Noisy Intermediate-Scale Quantum (NISQ) era, where restricted coherence times and high gate error rates require careful control of circuit design. As a result, hybrid dependencies and hardware-oriented constraints must often be handled manually, widening the gap between algorithmic intent and implementation and causing

design errors to be detected only at later stages, such as simulation or execution.

This limitation motivates the central problem addressed in this paper: the lack of explicit and systematic validation mechanisms at the modeling level for quantum circuits. In parallel, quantum technologies and cloud platforms have become more accessible [6], [7], [8], [9], [10], increasing the need for engineering methods that improve reliability before implementation.

In classical software engineering, similar complexity has been addressed through model-driven engineering (MDE), where metamodels define domain structure and constraints enforce well-formedness rules [11], [12], [13], [14], [15]. This combination improves analyzability, traceability, and early error detection [16], [17]. Related efforts in quantum software modeling have explored notations and intermediate representations [18], [19], but many proposals remain tightly coupled to specific frameworks or do not provide robust validation before execution [20].

Based on this gap, a constraint-based approach for quantum circuit modeling is proposed. A metamodel is used to represent core entities, including qubits, operations, measurements, and classical information flow, and a KCL (Kusion Configuration Language) [21] validation layer is introduced to specify and automatically check structural and behavioral rules [22]. This separation between structure and constraints aims to support reuse and extensibility across modeling scenarios.

To assess feasibility, the approach is evaluated using the quantum teleportation protocol as a case study [23]. The circuit is modeled and validated in both correct and intentionally faulty variants, allowing verification of whether constraint violations reveal design inconsistencies early. The main contributions are: (i) a metamodel for circuit-oriented quantum modeling, (ii) a KCL-based automated validation layer, and (iii) a feasibility study based on the teleportation protocol with eight fault injection scenarios.

The artifacts used in the evaluation, including the KCL specifications (`metamodel.k` and `constraints.k`) and the teleportation feasibility study models, are publicly available at <https://github.com/dsanmartins/Quantum-Constraint>. The repository also contains the mutated circuit models used in the fault injection scenarios, allowing the experiments to be reproduced.

The remainder of this paper is organized as follows. Section II provides background on quantum circuits and model-driven engineering concepts. Section III introduces the proposed metamodel for quantum circuit modeling. Section IV presents the constraint specification and validation mechanism. Section V describes the teleportation feasibility study and the evaluation with faulty circuit variants. Section VI presents related works. Section VII discusses limitations and possible extensions of the approach. Section VIII concludes the paper and outlines directions for future work.

II. BACKGROUND

A. Quantum Circuits

Quantum computing relies on computational models that differ fundamentally from classical computing paradigms. In classical systems, information is represented using bits that take deterministic values of either 0 or 1. In contrast, quantum computing employs *quantum bits* (qubits), which can exist in superpositions of the basis states $|0\rangle$ and $|1\rangle$. The state of a quantum system evolves according to the principles of quantum mechanics and is manipulated through quantum operations represented by unitary transformations [24].

Quantum algorithms are commonly described using the *quantum circuit model*. In this model, a computation is represented as a sequence of quantum gates applied to qubits arranged along circuit lines. Each wire corresponds to a qubit, and gates represent quantum operations that modify the state of the system. Single-qubit gates, such as the Hadamard or Pauli gates, act on individual qubits, while multi-qubit gates, such as the controlled-NOT (CNOT), create interactions between qubits and enable quantum entanglement [25].

In addition to unitary operations, quantum circuits include measurement operations that convert quantum states into classical outcomes. Measurement collapses the quantum state according to probabilistic rules and produces classical information that may be used for further computation or decision-making. As a result, quantum programs often combine quantum operations with classical control logic, especially in hybrid quantum-classical workflows [26].

Quantum circuits are typically represented as graphical diagrams where time progresses from left to right and operations are placed along the qubit wires. This representation provides an intuitive way to describe the structure of quantum algorithms and has been widely adopted in quantum programming frameworks. However, circuit diagrams alone do not provide explicit structural semantics for systematic software design, which motivates the exploration of higher-level modeling techniques for representing and validating quantum circuits [27], [28].

B. Model-Driven Engineering

Model-Driven Engineering (MDE) has been widely adopted as an approach for managing the complexity of software systems through the use of abstract models and automated transformations. In this paradigm, models are treated as primary development artifacts that describe the structure and

behavior of a system at different levels of abstraction. Central to MDE are *metamodels*, which define the abstract syntax and structural constraints of modeling languages, enabling the specification and validation of domain-specific models [29].

MDE techniques have also been explored in the context of quantum software development. As quantum applications increasingly interact with classical components, higher-level modeling approaches have been proposed to represent hybrid quantum-classical systems and to support software modernization processes. In particular, model transformations and architectural models have been investigated as mechanisms to represent quantum software artifacts and to facilitate the evolution and maintenance of quantum programs [30], [29].

The adoption of modeling techniques for quantum systems has also motivated the use of standard modeling languages such as UML. UML-based approaches enable quantum circuits and related software components to be described using well-established modeling constructs while preserving compatibility with existing software engineering tools and practices [31]. Such approaches provide a foundation for defining metamodels and validation rules capable of capturing the structural properties of quantum circuits, which motivates the model-based validation approach proposed in this work.

C. Constraint Languages and Model Validation

In model-driven engineering, constraints are commonly used to define well-formedness rules that cannot be expressed solely through the structural elements of a metamodel. While a metamodel defines the abstract syntax of a modeling language, constraint languages allow additional semantic restrictions to be specified and automatically verified. These constraints are typically used to enforce consistency conditions, validate model elements, and detect modeling errors early in the design process.

Several modeling approaches have incorporated constraint mechanisms to improve the reliability of software models. In the context of quantum software modeling, UML-based representations of quantum circuits have been proposed to describe the structural organization of qubits, gates, and classical control elements [27]. However, the correctness of such models often depends on additional domain-specific restrictions, such as the proper use of qubits, the consistency of gate configurations, and the validity of measurement dependencies. These restrictions cannot always be captured through the metamodel structure alone and therefore require explicit validation rules.

Constraint specifications have also been explored in the development and evolution of quantum software systems, where automated analysis tools can help detect inconsistencies and potential errors during the design phase [30]. By defining constraints that operate over model elements, it becomes possible to verify structural and behavioral properties of quantum circuits before their implementation in quantum programming frameworks. This capability motivates the use of a constraint layer in conjunction with the metamodel proposed in this work.

III. QUANTUM CIRCUIT METAMODEL

The design of the proposed metamodel is inspired by the modeling approach introduced in [27]. In particular, several structural concepts used to represent quantum circuits are adapted from that work. However, the metamodel presented here extends this representation by introducing explicit structural constraints and a validation layer aimed at supporting early verification of circuit models.

The section is organized as follows. First, the modeling requirements that guided the design of the metamodel are presented. Next, an overview of the metamodel elements and their relationships is described. Finally, the structural constraints that ensure the well-formedness of quantum circuit models are discussed.

A. Modeling Requirements

The proposed quantum circuit metamodel was designed to satisfy a set of requirements derived from the structural characteristics of circuit-based quantum computation and from the need for early validation in quantum software development. At a minimum, the metamodel must represent the core elements of quantum circuits, including qubits, quantum operations, and measurements, as well as the structural relationships between them. In particular, it must capture both the connectivity of circuit elements and the qubits on which operations act, so that the organization of the computation is preserved at the modeling level.

In addition, hybrid quantum–classical behavior must be represented explicitly. Since many quantum algorithms use measurement outcomes to control subsequent operations, the metamodel must include constructs for classical storage and for dependencies between measurements and classically controlled gates. The representation must also support structural validation through explicit constraints, allowing modeling errors such as invalid qubit references, inconsistent measurement outputs, or malformed gate definitions to be detected before implementation.

Finally, the metamodel should remain sufficiently abstract to support different quantum programming frameworks and hardware platforms. By providing a platform-independent representation of circuit elements and their relationships, it can serve as a basis for model validation and, potentially, for translation into multiple quantum software stacks.

B. Metamodel Definition

Figure 1 presents the metamodel used to represent quantum circuits in our approach. The design is inspired by the conceptual model proposed by Pérez-Castillo et al. [27], which uses a UML profile on top of Activity diagrams. However, our approach departs from UML extensions and adopts a standalone domain-specific modeling language with an explicit validation layer. In the current implementation, this metamodel is realized through KCL schemas that provide the structural basis for validation. At the root of the model, the `QuantumCircuit` class acts as the container for all structural elements. It includes an

ordering attribute (`OrderingKind`) that defines the qubit indexing convention.

Quantum and classical data are represented by the `Qubit` and `ClassicalStore` classes, respectively. Each `Qubit` has a `kind` attribute (`QubitKind`), whose values are summarized in Table I. In the current version of the metamodel, qubits are classified as either `data` or `ancilla`, while control is treated as a functional role assumed by a qubit within a gate rather than as an intrinsic qubit type.

The abstract class `Node` represents executable and control-flow elements of the circuit. It is specialized into operational nodes such as `GateNode`, `MeasureNode`, and `ResetNode`, as well as flow-structuring nodes such as `InitialNode`, `FinalNode`, `ForkNode`, and `JoinNode`. We define a Quantum Operation as an atomic functional unit within the execution flow that manipulates one or more qubits; in our metamodel, this abstract concept is realized through the operational node subtypes. To capture gate-level qubit roles, `GateNode` includes explicit `targets` and optional `controls` fields, allowing any declared qubit to participate as target or control depending on the operation. The execution structure is modeled through the `Flow` class, which connects nodes into a directed execution graph.

To ensure precision, several attributes are typed by enumerations. Table I summarizes these enumerated types, their admissible values, and their meaning in the quantum-circuit domain.

TABLE I
METAMODEL ENUMERATIONS AND DEFINITIONS

Enumeration	Values	Meaning
<code>QubitKind</code>	<code>data</code> , <code>ancilla</code>	The intrinsic lifecycle of the qubit (computational vs. auxiliary).
<code>BasisKind</code>	<code>X</code> , <code>Y</code> , <code>Z</code>	The computational basis used for measurement operations.
<code>OrderingKind</code>	<code>little</code> , <code>big</code>	The qubit indexing convention (endianness) for state representation.

The metamodel distinguishes between structural properties and higher-level semantics. For instance, while `MeasureNode` specifies a `basis` (defaulting to Z-basis), the consistency of this measurement with the subsequent classical control logic is handled by the behavioral constraints described in the following section.

C. Well-Formedness Constraints of the Quantum Circuit Model

The well-formedness constraints introduced in this work ensure that quantum circuit models are structurally and behaviorally consistent before execution or code generation. These constraints are derived from three complementary sources:

- structural requirements of circuit execution graphs,
- fundamental principles governing the interaction between quantum and classical information, and
- empirical observations on common modeling defects in quantum software.

Table II summarizes the eight constraints used in the proposed constraint-based validation approach. Together, these

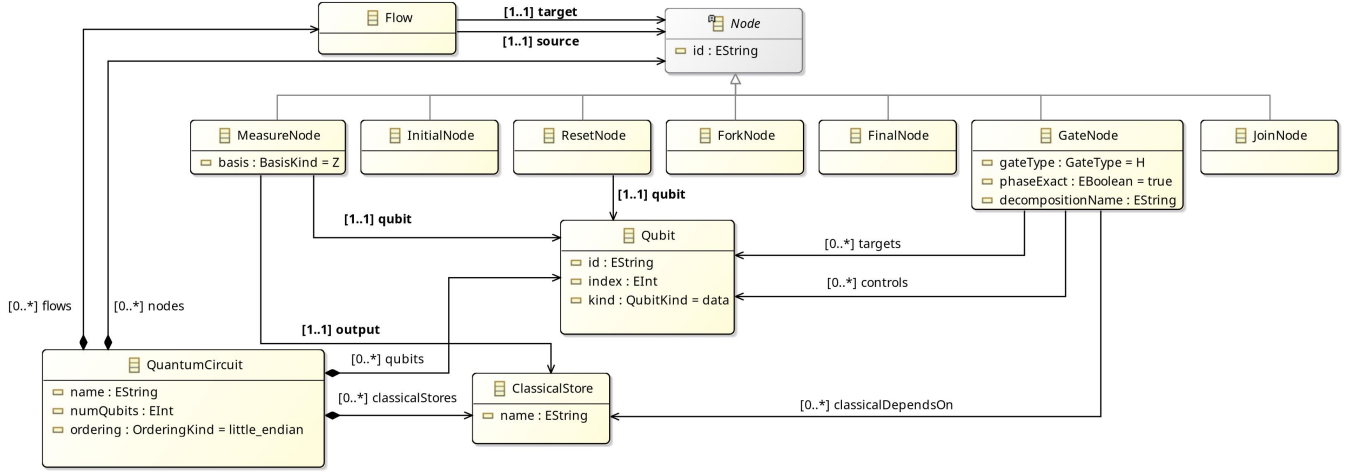


Fig. 1. Simplified metamodel for quantum circuit modeling. For presentation purposes, the abstract concept *Node* groups the executable circuit elements, while identifier-based references are validated through structural constraints.

rules define the minimum conditions that a circuit model must satisfy in order to be considered well formed. Structural constraints preserve the integrity of the circuit representation and its resource references, whereas behavioral constraints regulate the consistency of measurements and classical dependencies.

The first five constraints focus on graph integrity and resource consistency. C1 and C2 ensure that the execution flow is both well directed and semantically interpretable, excluding malformed or dangling flow relations. C3 and C4 regulate the internal validity of gate definitions by requiring that operations be attached to actual qubit resources and that controlled gates do not assign the same qubit simultaneously to incompatible roles. C5 complements these rules by ensuring that every referenced qubit or classical store is locally declared in the circuit being validated.

The remaining constraints govern the interaction between quantum operations and classical information. C6 enforces the consistency of measurement nodes by requiring that they consume a valid qubit and store their output in a valid classical memory element. C7 extends this requirement to hybrid control by checking that a classically controlled gate depends on a measurement outcome that is defined and found within the bounded predecessor traversal used by the current validator. Finally, C8 guarantees that the circuit possesses a well-formed execution skeleton from entry to exit, thereby preventing models with incomplete or improperly terminated control flow.

Unlike traditional static analysis tools that operate directly on source code (e.g., *LintQ* [32]), the proposed approach validates circuit models during the modeling phase. By enforcing these rules through the KCL engine, circuit instances such as the teleportation protocol presented in Section V can be checked automatically against their basic structural and hybrid-consistency requirements before translation into executable code. This reduces the risk of propagating design-level errors into later stages of quantum software development.

IV. CONSTRAINT-BASED VALIDATION

The well-formedness constraints introduced in the previous section define the structural and behavioral conditions that quantum circuit models must satisfy in order to represent valid quantum computations. However, defining such constraints at the metamodel level is not sufficient by itself. A mechanism is required to systematically evaluate whether a given model satisfies these constraints and to detect violations during the modeling process.

Constraint-based validation provides this mechanism by automatically verifying that models conform to the rules defined by the metamodel. In the context of quantum circuit modeling, this validation process ensures that circuit structures, qubit usage, and interactions between quantum and classical components remain consistent with the semantics of the quantum circuit model. By detecting violations early in the modeling phase, constraint-based validation helps prevent the propagation of errors to later stages of quantum program development. This section presents the validation architecture designed to enforce the well-formedness constraints defined for the proposed metamodel. The architecture specifies how circuit models are analyzed, how constraints are evaluated, and how validation results are reported to the user.

A. Validation Architecture

The validation architecture provides an automated mechanism for checking whether a quantum circuit model satisfies the well-formedness constraints defined for the metamodel. Following a constraint-based validation approach commonly used in model-driven engineering, a circuit model is analyzed by a validation engine that evaluates formal rules over the model elements.

Figure 2 illustrates the proposed architecture, which consists of three main elements: a quantum circuit model represented as an instance of the metamodel, a set of constraint definitions

TABLE II
WELL-FORMEDNESS CONSTRAINTS FOR THE QUANTUM CIRCUIT METAMODEL

ID	Type	Constraint Description	Rationale / Impact
C1	Structural	Flow Consistency: Flow relations must preserve the intended direction of execution; in particular, the <code>FinalNode</code> cannot be used as a source and the <code>InitialNode</code> cannot be used as a target.	Prevents malformed execution paths and preserves a meaningful start-to-end ordering of circuit actions.
C2	Structural	Node Existence: All identifiers referenced in flow relations must correspond to nodes declared in the same circuit.	Avoids references to undefined elements that would produce incomplete or non-interpretable circuit graphs.
C3	Structural	Qubit Binding: Every quantum operation must reference at least one qubit, either as a target or as a control.	Prevents empty operations that do not act on any physical resource and therefore do not correspond to valid quantum actions.
C4	Structural	Control-Target Disjointness: Control qubits and target qubits of a controlled gate must be disjoint.	Ensures the physical validity of controlled unitary operations such as CNOT and CZ gates.
C5	Structural	Resource Locality: Qubits and classical stores referenced by operations and measurements must belong to the circuit in which they are used.	Prevents cross-circuit references and enforces the internal consistency and modularity of circuit definitions.
C6	Behavioral	Measurement Integrity: A measurement must reference a valid qubit and write its result to a valid classical store declared in the same circuit.	Guarantees that the transfer of information from the quantum state to the classical layer is explicitly and consistently represented.
C7	Behavioral	Classical Dependency Safety: Classically controlled gates must depend only on classical values produced by valid preceding measurements in the execution flow.	Ensures causal consistency between measurement outcomes and the conditional gates that consume them.
C8	Structural	Control-Flow Anchoring: A circuit must include a properly anchored execution graph, including an outgoing flow from the <code>InitialNode</code> and an incoming flow to the <code>FinalNode</code> .	Guarantees that circuit execution has a well-defined entry and termination within the modeled flow graph.

expressed as KCL rules (C1–C8), and a validation engine that evaluates these rules over the model.

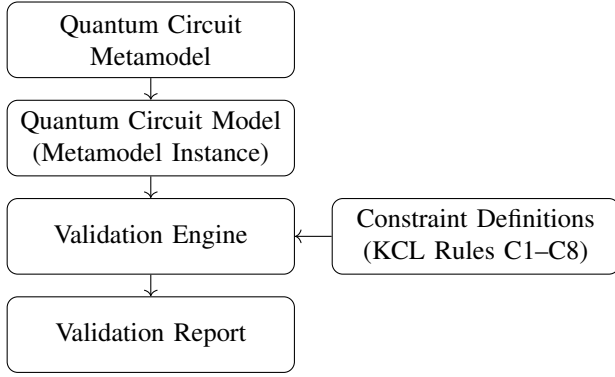


Fig. 2. Constraint-based validation architecture for quantum circuit models.

During validation, the engine evaluates the structural and behavioral constraints over the circuit model and produces a report whenever a violation is detected. The report identifies the violated rule together with the model element responsible for the inconsistency, supporting the early correction of design errors before translation into executable quantum code.

B. KCL as a Validation Engine

To operationalize the well-formedness constraints, the validation layer is implemented using KCL [21]. KCL is an open-source declarative configuration language with built-in validation capabilities, originally developed for policy and configuration management in cloud-native environments. Our approach defines abstract syntax through KCL schemas, which serve as the domain-specific language’s metaclasses. KCL’s schema-based structure and `check` mechanism make it suitable for model validation, since abstract syntax and invariant conditions can be expressed directly over structured data.

The implementation is organized into two complementary artifacts:

- `metamodel.k`: captures the abstract syntax of the quantum circuit language together with structural constraints, such as typing consistency, uniqueness of identifiers, and reference integrity;
- `constraints.k`: specifies higher-level validation rules governing measurement consistency, classical dependencies, and other execution-level conditions that complement the structural constraints defined in the metamodel.

This separation is intended to improve modularity and maintainability. Structural constraints are defined close to the metamodel because they are intrinsic to the language structure, whereas higher-level validation rules are specified independently. The choice of KCL over more traditional alternatives is also motivated by deployment considerations. While OCL [33] remains the standard companion to Ecore-based models, its use often depends on Eclipse-based toolchains. By contrast, KCL combines declarative validation, static typing, and platform-independent execution in a standalone binary, which facilitates integration into modern DevOps workflows and supports a shift-left strategy in which circuit models are verified during the design phase.

```

schema QuantumCircuit:
  numQubits: int
  qubits: [Qubit]
  gates?: [GateNode] = []
  # ... other fields (measures, flows, etc.)

check:
  len(qubits) == numQubits, "numQubits_must_match_declared_qubits"
  len({q.id for q in qubits}) == len(qubits), "Qubit_ids_must_be_unique"

  all(t in {q.id for q in qubits} for g in gates for
    t in g.targets), \
    "Every_gate_target_must_reference_an_existing_qubit"

  all(c in {q.id for q in qubits} for g in gates for
    c in g.controls), \
    "Every_gate_control_must_reference_an_existing_qubit"
  
```

Listing 1. Excerpt from `metamodel.k` showing structural validation rules in the `QuantumCircuit` schema

Constraints are expressed declaratively through `check` clauses associated with schema definitions. During validation, these conditions are evaluated over an instance of the quantum circuit metamodel, and an error is reported whenever a violation is found. Listing 1 illustrates representative structural checks in the `QuantumCircuit` schema, including consistency between the declared number of qubits and the circuit size, uniqueness of qubit identifiers, and validity of gate targets and controls. At the validation level, a dedicated schema named `CircuitValidation` is defined to evaluate additional consistency conditions not encoded directly in the metamodel. Listing 2 shows two representative rules: one for measurement-related consistency and another for classical dependency validation.

```

schema CircuitValidation:
  circuit: metamodel.QuantumCircuit

  check:
    # Measurement-related consistency
    all(
      m.qubit in {q.id for q in circuit.qubits} and
      m.output in {c.id for c in circuit.classicalStores}
    ), "Measurements_must_reference_valid_qubits_and_valid_classical_stores"

    # Classical dependency consistency
    all(
      d in {m.output for m in circuit.measures}
      for g in circuit.gates
      for d in g.classicalDependsOn
    ), "Classical_dependencies_must_reference_valid_measurement_outputs"

```

Listing 2. Excerpt from `constraints.k` showing validation rules in the `CircuitValidation` schema

Together with the remaining checks implemented across `metamodel.k` and `constraints.k`, these rules provide a concrete operationalization of the proposed well-formedness constraints. As a result, quantum circuit models can be validated automatically before code generation or execution, enabling early detection of modeling errors and improving the reliability of quantum software artifacts.

V. FEASIBILITY STUDY AND VALIDATION EXPERIMENTS

This section evaluates the proposed constraint-based validation approach through a feasibility study based on the quantum teleportation protocol. Quantum teleportation is a canonical protocol in quantum communication that enables the transfer of an unknown quantum state from one qubit to another using entanglement and classical communication. The teleportation circuit provides an appropriate benchmark for evaluating the proposed validation mechanism because it combines several fundamental elements of quantum programs, including entanglement generation, controlled quantum operations, measurements, classical information flow, and conditional correction gates. These features make it possible to exercise both structural and behavioral constraints defined for the quantum circuit metamodel.

Figure 3 shows the teleportation circuit modeled using the proposed metamodel representation. In addition to the quantum wires, the diagram includes elements that represent

classical communication channels. The symbol “+” denotes the classical interface, while `msg1` and `register1` represent classical storage elements that collect the outcomes of the measurements performed in the Z basis. These elements explicitly model the interaction between quantum operations and classical information, which is required for the conditional correction stage of the protocol.

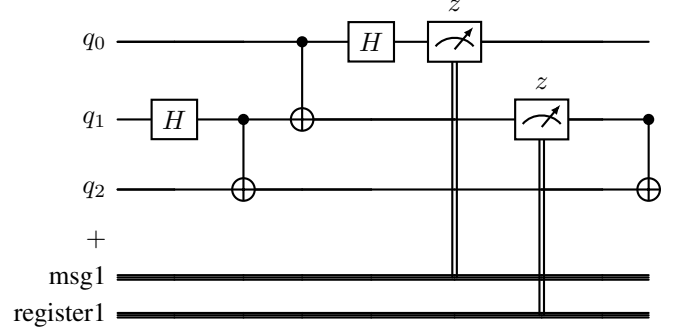


Fig. 3. Quantum teleportation circuit (from [27]) represented using the proposed quantum circuit metamodel

The goal of the experiments is to evaluate whether the constraints defined in the validation approach can detect modeling inconsistencies in circuit representations of the teleportation protocol.

A. Feasibility Study: Quantum Teleportation Protocol

The teleportation protocol transfers the quantum state of a source qubit to a target qubit through the use of an entangled pair and classical communication. The circuit involves three qubits. The first qubit (q_0) contains the quantum state to be teleported, while the remaining two qubits (q_1 and q_2) are used to generate an entangled pair that enables the transfer of the state.

The teleportation circuit proceeds through the following stages:

- 1) Generation of an entangled Bell pair between q_1 and q_2 by applying a Hadamard gate to q_1 followed by a controlled-NOT operation with q_1 as control and q_2 as target.
- 2) Interaction of the source qubit with the entangled pair through a controlled-NOT operation from q_0 to q_1 , followed by a Hadamard gate applied to q_0 .
- 3) Measurement of qubits q_0 and q_1 in the Z basis, producing two classical outcomes that are transmitted through the classical interface.
- 4) Application of correction gates on the target qubit q_2 . Depending on the measurement outcomes, a Z gate and an X gate are applied to reconstruct the original quantum state on the target qubit.

B. Experimental Setup

The goal of the study is to assess whether the proposed constraint-based validation approach can detect modeling errors in quantum circuit representations. The experiments were

conducted using the teleportation circuit presented in Figure 3, which was modeled as an instance of the quantum circuit metamodel introduced in Section III-B.

The validation process relies on the set of eight well-formedness constraints defined in Section III. These constraints capture both structural properties of the circuit graph and behavioral relationships between quantum operations and classical information produced by measurements.

The circuit model was specified in KCL using the meta-model definition and the constraint rules provided in the .k specification files. The constraints were implemented using KCL rules that are evaluated by the validation engine during model analysis.

To evaluate the effectiveness of the proposed approach, two types of circuit models were analyzed:

- 1) **Correct model.** A correct implementation of the teleportation protocol that satisfies all structural and behavioral constraints.
- 2) **Faulty model.** A modified version of the circuit containing intentional modeling errors designed to violate specific constraints.

Each fault scenario was implemented as an independent mutated model derived from the base teleportation circuit. This design ensures that each constraint violation can be evaluated in isolation without interference from other injected faults.

The faulty model introduces representative modeling mistakes that may occur during circuit design, such as incorrect gate connections, invalid control relations, or inconsistent measurement dependencies. These errors allow the validation engine to exercise the defined constraints and verify whether violations are properly detected.

During validation, the engine traverses the circuit model and evaluates the constraint rules over the corresponding model elements. When a constraint is not satisfied, the validation process reports the violation and identifies the element responsible for the inconsistency. The generated validation report allows the modeler to locate and correct errors in the circuit design.

This experimental setup allows us to evaluate the ability of the proposed approach to automatically detect modeling inconsistencies in quantum circuits before execution, contributing to improving the reliability of quantum software artifacts.

C. Fault Injection Scenarios

To evaluate the effectiveness of the proposed validation approach, we conducted a set of experiments based on fault injection principles, a technique closely related to mutation testing [34], [35]. In this setting, fault injection consists of deliberately introducing small structural or semantic modifications into a correct model in order to assess whether the validation rules can detect the resulting inconsistencies. Following the guidelines discussed by Jia and Harman [35], the injected faults were designed to simulate representative modeling errors that may occur during the design of quantum circuits.

The experiments compare a valid specification (teleportation.k) against independently mutated

faulty models defined in `teleportation_faulty.k`. Each faulty scenario was constructed as a separate mutant derived from the same base teleportation circuit, so that each intended violation could be analyzed in isolation. In the current implementation, eight fault scenarios are defined, each targeting one of the eight well-formedness constraints introduced in Section III-C.

- 1) **Flow Consistency Violation (F1, violates C1):** A malformed flow relation is introduced so that the execution graph violates the intended direction of computation, for example by using the `FinalNode` as a source or the `InitialNode` as a target. This scenario checks whether the validation layer rejects invalid execution paths.
- 2) **Node Existence Violation (F2, violates C2):** A flow relation references a node identifier that is not declared in the circuit. This scenario simulates an incomplete or inconsistent graph specification and verifies that all flow endpoints correspond to existing nodes.
- 3) **Qubit Binding Violation (F3, violates C3):** A gate is declared without any associated qubit, that is, without targets and without controls. This mutation tests whether the validation engine rejects operations that are not bound to any quantum resource.
- 4) **Control-Target Conflict (F4, violates C4):** A controlled gate is defined so that the same qubit appears simultaneously as control and target. This scenario verifies that the validation rules enforce the physical consistency of controlled operations.
- 5) **Resource Locality Violation (F5, violates C5):** A gate or measurement references a qubit or classical store that does not belong to the circuit in which it is used. This scenario checks whether the validation layer prevents non-local or cross-circuit references.
- 6) **Measurement Integrity Violation (F6, violates C6):** A measurement is defined with an invalid classical output, for instance by writing its result to a classical store that is not declared in the circuit. This scenario verifies that measurement nodes correctly connect the quantum and classical layers of the model.
- 7) **Classical Dependency Safety Violation (F7, violates C7):** A classically controlled gate depends on a classical value that is not associated with any valid preceding measurement under the bounded dependency analysis performed by the current validator. Listing 3 illustrates this kind of fault, where the gate references a classical store that exists syntactically but is not matched to a prior measurement within the implemented search scope.
- 8) **Control-Flow Anchoring Violation (F8, violates C8):** The execution graph is mutated so that the circuit loses its proper anchoring, for example by removing the flow that should reach the `FinalNode`. This scenario verifies that the model retains a well-defined entry-to-exit execution structure.

The faulty scenarios were intentionally designed to main-

tain the overall teleportation structure while introducing one primary inconsistency per mutant. This design makes the relationship between injected fault, violated constraint, and validation outcome explicit, thereby improving the traceability of the empirical evaluation.

```
# Mutation: gate depends on a classical value not produced
# by any preceding measurement in the execution flow
gates = [
    metamodel.GateNode {
        id = "g5", gateType = "X", targets = ["q2"],
        classicalDependsOn = ["cX"] # Error: cX is never
        produced
    }
]
```

Listing 3. Faulty classical dependency in the teleportation circuit

To illustrate the practical utility of the approach, Listing 4 shows the diagnostic output produced by the KCL compiler when validating the faulty circuit scenario **F7**, which corresponds to a violation of classical dependency safety.

```
error[E2L23]: CompileError
--> teleportation_faulty.k:118:1
| validation_f7 = constraints.CircuitValidation {
|   circuit = teleportation_faulty_f7
| }
Check failed: C7 Violation: Classical dependency refers to
a measurement not yet performed
```

Listing 4. KCL validation report for Scenario F7 (Classical Dependency Safety violation)

The report identifies the exact location of the violation and associates it with the corresponding formal constraint, demonstrating that the validation mechanism provides precise design-time diagnostic feedback.

D. Validation Results

The validation experiments were conducted by executing the constraint rules defined in `constraints.k` over both the correct and faulty teleportation circuit models. For the correct circuit specification (`teleportation.k`), all constraints were satisfied and no violations were reported. This confirms that the base teleportation circuit conforms to the structural and behavioral requirements defined by the proposed metamodel and validation layer.

For the faulty specifications contained in `teleportation_faulty.k`, the validation engine successfully detected the intended inconsistency in each mutated scenario. Table III summarizes the injected faults and their mapping to the corresponding well-formedness constraints.

The results show that the proposed constraint-based validation approach can detect a diverse set of modeling inconsistencies in quantum circuit specifications. Structural constraints capture errors related to graph organization, node references, gate-resource bindings, and flow anchoring, whereas behavioral constraints identify incorrect interactions between measurement outcomes and classically controlled operations.

These findings indicate that the proposed approach can detect design-level errors before circuit execution or translation into executable code. Such early validation contributes to improving the reliability of quantum software artifacts and

supports the construction of correct circuit models during the modeling phase.

E. Threats to Validity

Internal validity. The fault scenarios were designed by the same authors who defined the constraints, which may introduce selection bias toward detectable errors. In addition, false negatives (errors not captured by constraints C1–C8) were not systematically evaluated.

External validity. The evaluation uses a single benchmark protocol. Results may not generalize to other families of quantum programs such as variational algorithms, error-correcting codes, or large-scale optimization circuits.

Construct validity. Error detection is operationalized as violations of KCL constraints. No user study was conducted to evaluate whether the resulting validation reports are easily interpretable by quantum software engineers.

VI. RELATED WORK

Research in quantum software engineering has recently begun to address the challenges of designing, validating, and debugging quantum programs. Existing work can be broadly grouped into three areas: (i) modeling approaches for quantum software, (ii) empirical studies of quantum software bugs, and (iii) static analysis techniques for fault detection.

A. Modeling Approaches for Quantum Software

Several studies have explored the use of software engineering modeling techniques to support quantum system design. Pérez-Castillo et al. propose a UML-based approach in which quantum circuits are modeled using extensions of UML activity diagrams [27]. Their UML profile introduces domain concepts such as qubits, quantum gates, and measurement operations as stereotypes, allowing quantum circuits to be represented using high-level abstractions and integrated with classical software design models.

While these approaches provide useful visual representations, they mainly serve as documentation artifacts and do not enforce structural correctness automatically. In contrast, our work defines a textual metamodel with constraint-based validation, enabling automated verification of circuit structure and consistency during the modeling stage.

B. Studies and Taxonomies of Quantum Software Bugs

Empirical studies have investigated the types of faults that occur in quantum software. Zhao et al. analyzed bug patterns in Qiskit programs and classified them according to stages such as initialization, gate operations, and measurement [36]. Similarly, Quetschlich and Di Matteo provide an experience-based taxonomy of quantum software bugs, highlighting issues related to quantum-specific phenomena such as superposition and entanglement [37].

These studies provide valuable insights into common sources of errors. Our work builds on these findings by encoding representative bug patterns as formal constraints, enabling the detection of such errors during the modeling phase.

TABLE III
VALIDATION RESULTS FOR THE INJECTED FAULT SCENARIOS AND THEIR MAPPING TO WELL-FORMEDNESS CONSTRAINTS

ID	Error Type	Injected Fault Description	Violated Constraint
F1	Flow consistency	Invalid flow relation violating the execution direction of the circuit graph.	C1
F2	Node existence	Flow relation references an undefined node identifier.	C2
F3	Qubit binding	Gate declared without targets or controls, hence not bound to any qubit resource.	C3
F4	Control–target conflict	Controlled gate uses the same qubit as both control and target.	C4
F5	Resource locality	Operation references a qubit or classical store not declared in the circuit.	C5
F6	Measurement integrity	Measurement writes its output to an invalid or undeclared classical store.	C6
F7	Classical dependency safety	Classically controlled gate depends on a classical value not produced by a valid preceding measurement.	C7
F8	Control-flow anchoring	Execution graph lacks proper anchoring, e.g., no valid flow reaches the <code>FinalNode</code> .	C8

C. Static Analysis of Quantum Programs

Static analysis tools aim to identify faults in quantum programs before execution. For example, *LintQ* analyzes Qiskit programs by introducing abstractions for circuits, gates, and qubits and detecting issues such as redundant measurements or invalid gate usage [32].

Unlike static analysis approaches that operate on source code, our method performs validation at the model level. By integrating constraint-based checks into the design phase, errors can be detected earlier in the development lifecycle before code generation or implementation.

VII. DISCUSSION

This work explores the use of model-driven engineering (MDE) principles to support the design and validation of quantum circuits through a constraint-based approach. The proposal combines a domain-specific metamodel with a set of well-formedness constraints implemented in KCL. A key advantage of this approach is the separation between the structural representation of circuits and the validation logic, allowing validation rules to evolve independently from the metamodel.

By introducing validation at the modeling stage, the approach supports a “shift-left” strategy for quantum debugging [38]. Unlike formal verifiers (e.g., QHL) that provide exhaustive correctness guarantees through semantic reasoning, our approach operates at the lightweight, design-time end of the validation spectrum. By treating constraints like C6 and C7 as well-formedness rules, inconsistencies can be identified during the design phase. The fault injection scenarios (F1–F8) indicate that the validation layer effectively detects typical modeling errors such as invalid circuit connections and incorrect quantum–classical dependencies.

Although the teleportation protocol is a relatively small benchmark, it captures essential hybrid quantum–classical features such as entanglement, measurement, and conditional control. Since each rule (C1–C8) is evaluated over circuit elements and their local relationships, the same constraint definitions can in principle be applied to larger circuit models without modification. However, the current evaluation is limited to a single case study, and further experiments on larger and more diverse benchmarks are needed to assess the practical scalability of the approach. The proposal also complements industry standards such as OpenQASM [39]. While OpenQASM focuses on hardware-level instructions, the

metamodel supports higher-level design-time validation before circuits are translated into executable code.

Despite these advantages, several limitations remain. The current constraints focus primarily on structural correctness and local behavioral consistency, leaving advanced properties such as circuit equivalence or algorithm-level verification for future work. Additionally, further experiments with larger and more diverse quantum circuits are necessary to evaluate performance limits in complex scenarios.

VIII. CONCLUSION

This paper presented a constraint-based modeling approach for the representation and validation of quantum circuits within a model-driven engineering context. The proposed approach introduces a domain-specific metamodel that captures the fundamental structural elements of quantum circuits, including qubits, operations, measurements, classical storage, and execution flow, complemented by a validation layer that enforces well-formedness rules. Unlike traditional programming APIs where errors are often only caught during execution, this work provides a structured abstraction to reason about circuit correctness at the design level.

The validation mechanism, implemented in KCL, enables the automatic verification of structural and behavioral rules. This allows modeling errors to be detected during the design phase, supporting a shift-left strategy that identifies inconsistencies before circuits are translated into executable code or dispatched to expensive quantum hardware. By externalizing validation logic from the metamodel, the approach promotes modularity, allowing rules to evolve independently as new quantum computing constraints or hardware limitations emerge.

The evaluation based on the quantum teleportation protocol and the injected fault scenarios shows that the validation mechanism successfully detected the intended inconsistency in each mutated model. The detected faults include invalid flow relations, undefined node references, qubit-binding errors, control–target conflicts, non-local resource references, invalid measurement outputs, unsafe classical dependencies, and improper control-flow anchoring. These findings suggest that incorporating constraint-based validation into the modeling lifecycle can strengthen the reliability of quantum software artifacts by enabling the early detection of design-level errors in hybrid quantum–classical circuits.

As future work, the approach will be extended in three directions: interoperability through Model-to-Text transformations

toward executable formats such as OpenQASM 3.0 and Qiskit, together with IDE integration for real-time feedback; richer validation through hardware-aware constraints and performance evaluation on larger benchmarks; and closer integration with formal verification techniques to support algorithm-level assertions and state equivalence checks. Overall, this work contributes to the professionalization of quantum software development by supporting the early detection of structural and behavioral modeling errors in hybrid quantum–classical systems.

REFERENCES

- [1] C. Columbus Chinnappan, P. Thanaraj Krishnan, E. Elamaram, R. Arul, and T. Sunil Kumar, “Quantum computing: Foundations, architecture and applications,” *Engineering Reports*, vol. 7, no. 8, p. e70337, 2025.
- [2] D. K. Srimannaryana, “From quantum mechanics to quantum advantage: A comprehensive survey of theoretical foundations, algorithmic innovations, and hardware implementations in the nisq era,” *International Journal of All Research Education and Scientific Methods*, vol. 13, no. 03, p. 4068–4075, 2025.
- [3] V. Stirbu, M. Haghparsat, M. Waseem, N. Dayama, and T. Mikkonen, “Full-stack quantum software in practice: Ecosystem, stakeholders and challenges,” in *2023 IEEE International Conference on Quantum Computing and Engineering (QCE)*. IEEE, 9 2023, p. 177–180.
- [4] P. Selinger, “Challenges in quantum programming languages (invited talk),” in *3rd International Conference on Formal Structures for Computation and Deduction (FSCD 2018)*, vol. 108. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2018, pp. 3:1–3:2.
- [5] R. Seidel, S. Bock, R. Zander, M. Petrič, N. Steinmann, N. Tcholtchev, and M. Hauswirth, “Qrisp: A framework for compilable high-level programming of gate-based quantum computers,” 2024.
- [6] A. Galindo and M. A. Martín-Delgado, “Information and computation: Classical and quantum aspects,” *Reviews of Modern Physics*, vol. 74, pp. 347–423, 1998.
- [7] L. Henriot, L. Beguin, A. Signoles, T. Lahaye, A. Browaeys, G.-O. Raymond, and C. Jurczak, “Quantum computing with neutral atoms,” *Quantum*, vol. 4, p. 327, 9 2020.
- [8] J. Preskill, “Quantum computing and the entanglement frontier,” 2012.
- [9] R. LaRose, “Overview and comparison of gate level quantum software platforms,” *Quantum*, vol. 3, p. 130, 3 2019.
- [10] A. Ahmad, A. B. Altamimi, and J. Aqib, “A reference architecture for quantum computing as a service,” *Journal of King Saud University - Computer and Information Sciences*, vol. 36, no. 6, p. 102094, 2024.
- [11] D. C. Schmidt, “Model-driven engineering,” *Computer*, vol. 39, no. 2, pp. 25–31, 2006.
- [12] J. Di Rocco, D. Di Ruscio, C. Di Sipio, P. T. Nguyen, and R. Rubel, “On the use of large language models in model-driven engineering,” *Software and Systems Modeling*, vol. 24, no. 3, pp. 923–948, Jun 2025.
- [13] J. Bézin, “On the unification power of models,” *Software and Systems Modeling*, vol. 4, no. 2, pp. 171–188, 2005.
- [14] M. Brambilla, J. Cabot, and M. Wimmer, *Model-Driven Software Engineering in Practice*. Morgan & Claypool, 2017.
- [15] J. Warmer and A. Kleppe, *The Object Constraint Language: Getting Your Models Ready for MDA*. Addison-Wesley, 2003.
- [16] W. Zhang, J. Holtmann, D. Strüder, R. Hebig, and J.-P. Steghöfer, “Supporting meta-model-based language evolution and rapid prototyping with automated grammar transformation,” *Journal of Systems and Software*, vol. 214, p. 112069, 2024.
- [17] A. Markaj, F. Gehlhoff, and A. Fay, “Semantic-aware validation in model-driven requirements engineering using shacl,” in *Proceedings of the 16th International Joint Conference on Knowledge Discovery, Knowledge Engineering and Knowledge Management - KEOD, IN-STICC*. SciTePress, 2024, pp. 191–198.
- [18] M. Soeken, T. Häner, and M. Roetteler, “Programming quantum computers using design automation,” 2018.
- [19] T. Häner, D. S. Steiger, K. Svore, and M. Troyer, “A software methodology for compiling quantum programs,” *Quantum Science and Technology*, vol. 3, no. 2, p. 020501, 3 2018.
- [20] X. Fu, J. Yu, X. Su, H. Jiang, H. Wu, F. Cheng, X. Deng, J. Zhang, L. Jin, Y. Yang, L. Xu, C. Hu, A. Huang, G. Huang, X. Qiang, M. Deng, P. Xu, W. Xu, W. Liu, Y. Zhang, Y. Deng, J. Wu, and Y. Feng, “Quingo: A programming framework for heterogeneous quantum-classical computing with nisq features,” *ACM Transactions on Quantum Computing*, vol. 2, no. 4, p. 1–37, 12 2021.
- [21] X. Duo, P. Xu, Z. Zhang, S. Chai, R. Xia, and Z. Zong, “Kcl: A declarative language for large-scale configuration and policy management,” in *Dependable Software Engineering. Theories, Tools, and Applications: 8th International Symposium, SETTA 2022, Beijing, China, October 27–29, 2022, Proceedings*. Berlin, Heidelberg: Springer-Verlag, 2022, p. 88–105.
- [22] N. C. Leite Ramalho, H. Amario de Souza, and M. Lordello Chaim, “Testing and debugging quantum programs: The road to 2030,” *ACM Transactions on Software Engineering and Methodology*, vol. 34, no. 5, p. 1–46, 4 2025.
- [23] C. H. Bennett, G. Brassard, C. Crépeau, R. Jozsa, A. Peres, and W. K. Wootters, “Teleporting an unknown quantum state via dual classical and einstein-podolsky-rosen channels,” *Phys. Rev. Lett.*, vol. 70, pp. 1895–1899, Mar 1993.
- [24] A. Montanaro, “Quantum algorithms: an overview,” *npj Quantum Information*, vol. 2, no. 1, p. 15023, 1 2016.
- [25] A. Barenco, C. H. Bennett, R. Cleve, D. P. DiVincenzo, N. Margolus, P. Shor, T. Sleator, J. A. Smolin, and H. Weinfurter, “Elementary gates for quantum computation,” *Phys. Rev. A*, vol. 52, pp. 3457–3467, Nov 1995.
- [26] J. Preskill, “Quantum Computing in the NISQ era and beyond,” *Quantum*, vol. 2, p. 79, Aug. 2018.
- [27] R. Pérez-Castillo, L. Jiménez-Navajas, and M. G. Piattini, “Modelling quantum circuits with uml,” *2021 IEEE/ACM 2nd International Workshop on Quantum Software Engineering (Q-SE)*, pp. 7–12, 2021.
- [28] R. Pérez-Castillo and M. Piattini, “Quantum software engineering: current state and challenges,” *Applied Sciences*, 2023.
- [29] L. Jiménez-Navajas, R. Pérez-Castillo, and M. Piattini, “Kdm to uml model transformation for quantum software modernization,” in *Quality of Information and Communications Technology*, A. C. R. Paiva, A. R. Cavalli, P. Ventura Martins, and R. Pérez-Castillo, Eds. Cham: Springer International Publishing, 2021, pp. 211–224.
- [30] J. A. Cruz-Lemus and M. A. Serrano, *Quantum Software Tools Overview*. Cham: Springer International Publishing, 2022, pp. 229–250.
- [31] R. Pérez-Castillo and M. Piattini, “Design of classical-quantum systems with uml,” *Computing*, vol. 104, no. 11, pp. 2375–2403, Nov 2022.
- [32] M. Paltenghi and M. Pradel, “Analyzing quantum programs with lintq: A static analysis framework for qiskit,” *Proc. ACM Softw. Eng.*, vol. 1, no. FSE, Jul. 2024.
- [33] J. Warmer and A. Kleppe, “Introduction to ocl,” in *Proceedings of the Technology of Object-Oriented Languages and Systems*, ser. TOOLS ’99. USA: IEEE Computer Society, 1999, p. 405.
- [34] R. A. DeMillo, R. J. Lipton, and F. G. Sayward, “Hints on test data selection: Help for the practicing programmer,” *Computer*, vol. 11, no. 4, p. 34–41, 4 1978.
- [35] Y. Jia and M. Harman, “An analysis and survey of the development of mutation testing,” *IEEE Trans. Softw. Eng.*, vol. 37, no. 5, p. 649–678, 9 2011.
- [36] P. Zhao, J. Zhao, and L. Ma, “Identifying bug patterns in quantum programs,” in *2021 IEEE/ACM 2nd International Workshop on Quantum Software Engineering (Q-SE)*, 2021, pp. 16–21.
- [37] N. Quetschlich and O. Di Matteo, “An experience-based classification of quantum bugs in quantum software,” *Computing*, vol. 107, no. 10, p. 193, Sep 2025.
- [38] V. S. Rani, D. A. R. Babu, K. Deepthi, and V. R. Reddy, “Shift-left testing in devops: A study of benefits, challenges, and best practices,” in *2023 2nd International Conference on Automation, Computing and Renewable Systems (ICACRS)*, 2023, pp. 1675–1680.
- [39] A. Cross, A. Javadi-Abhari, T. Alexander, N. De Beaudrap, L. S. Bishop, S. Heidel, C. A. Ryan, P. Sivarajah, J. Smolin, J. M. Gambetta, and B. R. Johnson, “Openqasm 3: A broader and deeper quantum assembly language,” *ACM Transactions on Quantum Computing*, vol. 3, no. 3, 9 2022.